VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

REPORT PROGRESS

12/04/2026

MACHINE LEARNING

GROUP 11

Supervisors: Ph. D Truong Vinh Lan

o0o

Students: Tran Dang Khoa - 2252363
Phan Tran Quoc Tuan - 2353273
Tran Gia Lam - 2352670
Nguyen Tan Hung - 2352439
Nguyen Huu Cau - 2352129

HO CHI MINH CITY, APRIL 12 2026

Contents

1 Project Overview **2**

2 Introduction **2**
2.1 Problem Description 2
2.2 Objective 2

3 EDA Overview **2**

4 Random Forest Model **3**
4.1 What is a Random Forest? 3
4.2 How It Works 3
4.3 Justification for Random Forest 3

5 Model Parameters **4**

6 Hyperparameter Tuning with Optuna **4**
6.1 Overview of Optuna 4
6.2 Objective Function 4
6.3 Search Space 5
6.4 Tuning Strategy 5

7 Final Model Configuration **5**
7.1 Best Hyperparameters Found 5
7.2 Preprocessing Pipeline 5
7.3 Final Model Performance 6

8 Conclusion **6**
8.1 Summary of Results 6
8.2 Future Work 6

1 Project Overview

- **Topic:** Assignment 1 - Machine Learning with Tabular Data
- **Project's name:** Home's Value Prediction
- **GitHub Repository:** [Group 11 - GitHub Repository]

Random Forest Modelling with Optuna: Throughout two weeks up to April 12, our group moved from analysis to modelling, building directly on the insights gained during EDA. A baseline Random Forest Regressor was trained to establish an initial performance benchmark. We also do the optimisation using the Optuna framework was subsequently applied over 30 trials to improved performance. The results and interpretation will be presented in the report.

2 Introduction

2.1 Problem Description

This report addresses the Zillow Prize Kaggle challenge, in which the objective is to improve Zillow's proprietary valuation model, commonly known as the *Zestimate*. The Zestimate produces an automated estimate of residential property values across the United States. Despite its widespread use, the model is not perfect, and its residual error – referred to as the *log-error* – contains patterns that can be exploited by a secondary predictive model.

The log-error is formally defined as:

$$\text{logerror} = \log(\text{Zestimate}) - \log(\text{SalePrice}) \tag{1}$$

A positive log-error indicates that the Zestimate overestimates the true sale price, while a negative value indicates underestimation. The magnitude of the log-error reflects the degree of inaccuracy of the original Zestimate.

2.2 Objective

The primary objective of this task is to build a supervised regression model that predicts the `logerror` for residential properties across the County based on a set of property attributes. The dataset covers transaction records from 2016 and 2017.

This is a **regression problem**: the target variable `logerror` is a continuous real-valued quantity with a mean of approximately 0.014 and a standard deviation of 0.166, exhibiting a distribution that is approximately Gaussian with scale ranging from -4.66 to $+5.26$.

The modelling approach adopted in this report is the **Random Forest Regressor**, supplemented by **Optuna** for hyperparameter optimisation. Performance is evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2).

3 EDA Overview

AnData Analysis (EDA) was conducted in the assignment and is not reproduced in detail here. The key findings that informed the modelling pipeline are summarised below.

- **Dataset size:** The final merged dataset consists of 167,888 transaction records spanning 2016 and 2017, with 61 original columns covering structural property attributes, geographical identifiers, tax assessments, and transaction metadata.
- **Target distribution:** The target variable `logerror` is approximately around zero (mean ≈ 0.014).
- **Missing values:** Thirteen columns exceed a 98% missing rate (e.g., `basementsqft`, `buildingclasstypeid`, `storytypeid`) and were removed.
- **Feature types:** The dataset contains 53 numeric and 6 object-type (categorical) columns. Numeric columns span a wide range of scales. Categorical columns include high-cardinality fields such as `propertycountylandusecode` and `propertyzoningdesc`.

- **Correlations:** Individual feature correlations with `logerror` are generally low (absolute values < 0.05), suggesting that the log-error signal is diffuse and that ensemble methods capable of capturing non-linear interactions are well-suited to the task.
- **Outliers:** Multiple numeric columns exhibit significant outlier proportions (e.g., `garagecarcnt` at 26.8%, `roomcnt` at 22.8%), during preprocessing.

4 Random Forest Model

4.1 What is a Random Forest?

A Random Forest is an ensemble learning method based on the aggregation of multiple decision trees. It was introduced by *Breiman (2001)* and operates under the principle of *bagging* (Bootstrap Aggregating) combined with random feature subspace selection. Each individual tree in the forest is trained on a bootstrapped sample of the training data and considers only a random subset of features at each split node.

For a regression task, the prediction of the Random Forest is the average of the predictions of all constituent trees:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} \hat{y}_t(\mathbf{x}) \tag{2}$$

where T is the number of trees and $\hat{y}_t(\mathbf{x})$ is the prediction of the t-th tree for input $\mathbf{x}$.

4.2 How It Works

The training procedure for a Random Forest consists of the following steps:

1. For each tree $t = 1, \ldots, T$:
   (a) Draw a bootstrap sample $\mathscr{D}_t$ of size n with replacement from the training set.
   (b) Grow a decision tree on $\mathscr{D}_t$. At each node, select the best split from a random subset of m features (where $m \ll p$, the total number of features).
   (c) Continue splitting until a stopping criterion is met (e.g., minimum samples per leaf, maximum tree depth).
2. At inference time, aggregate predictions across all trees by averaging (for regression).

4.3 Justification for Random Forest

The Random Forest is well-suited for this problem for several reasons:

- **High dimensionality:** After preprocessing, the feature space contains 49 input variables. Random Forests naturally handle high-dimensional data without requiring explicit feature selection.
- **Mixed feature types:** The dataset contains a mixture of numeric and encoded categorical features. Decision tree-based models are agnostic to feature scale, reducing the sensitivity to imperfect standardisation.
- **Non-linear relationships:** The correlations observed in the EDA are weak and linear models would likely underfit. Random Forests can capture complex, non-linear interactions between features.
- **Robustness to outliers:** The ensemble nature of the model and the use of split-based criteria make it more robust to residual outliers than linear regressors.
- **Built-in feature importance:** Random Forests provide impurity-based feature importance scores, enabling interpretability and potential feature engineering in subsequent iterations.

5 Model Parameters

Hyperparameter	Type	Role and Impact
`n_estimators`	Integer	Number of trees in the forest. More trees generally improve stability and reduce variance.
`max_depth`	Integer / None	Maximum depth of each tree. Limits model complexity.
`min_samples_split`	Integer	Minimum number of samples required to split an internal node.
`min_samples_leaf`	Integer	Minimum number of samples required in a leaf node.
`max_features`	String / Float	Number of features considered at each split. Controls the degree of randomness including `sqrt`, `log2`, or a fixed fraction.
`bootstrap`	Boolean	Whether to use bootstrap sampling when building trees. When `True`, each tree is trained on a random draw with replacement, enabling out-of-bag error estimation.
`random_state`	Integer	Seed for reproducibility. Fixed at 42 throughout the experiment.
`n_jobs`	Integer	Number of CPU cores to use for parallel training. Set to -1 to utilise all available cores.

Table 1: *Key Hyperparameters of the Random Forest Regressor*

The baseline model was initialised with `n_estimators=100` and all remaining parameters set to their `scikit-learn` defaults. This configuration served as the performance reference before hyperparameter optimisation.

6 Hyperparameter Tuning with Optuna

6.1 Overview of Optuna

Optuna is an open-source, automatic hyperparameter optimisation framework that implements a *define-by-run* interface. Unlike grid or random search, Optuna employs Bayesian optimisation via Tree-structured Parzen Estimator (TPE) sampling, which builds a probabilistic model of the objective function and intelligently focuses sampling in promising regions of the hyperparameter space.

6.2 Objective Function

The objective function passed to Optuna evaluates a candidate hyperparameter configuration by training a Random Forest on the training set and returning the 3-fold cross-validated Mean Absolute Error (MAE):

$$\text{Objective}(\theta) = \frac{1}{K}\sum_{k=1}^{K} \text{MAE}(y_k, \hat{y}_k(\theta)) \tag{3}$$

where $K = 3$ is the number of cross-validation folds and θ denotes the hyperparameter configuration under evaluation. The study was configured with `direction="minimize"`, instructing Optuna to seek configurations that minimise the CV MAE.

6.3 Search Space

Hyperparameter	Search Type	Range / Choices
n_estimators	Integer (step 50)	[50, 300]
max_depth	Integer	[3, 20]
min_samples_split	Integer	[2, 20]
min_samples_leaf	Integer	[1, 10]
max_features	Categorical	{"sqrt", "log2", 0.3, 0.5}
bootstrap	Categorical	{True, False}

Table 2: *Optuna Hyperparameter Search Space*

6.4 Tuning Strategy

The study was named `rf_zillow` and executed for **30 trials**. At each trial, Optuna sampled a configuration from the search space using its default TPE sampler. The full training set (`X_train_processed_df`, 134,310 samples) was used for cross-validation to ensure a reliable estimate of generalisation performance. Parallelism was enabled via `n_jobs=-1` both at the estimator and the cross-validation level to minimise wall-clock time.

After all 30 trials completed, the study identified Trial #20 as the best configuration, achieving a cross-validated MAE of **0.068840**.

7 Final Model Configuration

7.1 Best Hyperparameters Found

Hyperparameter	Best Value
n_estimators	300
max_depth	8
min_samples_split	10
min_samples_leaf	9
max_features	log2
bootstrap	True
random_state	42
n_jobs	−1

Table 3: *Best Hyperparameters Identified by Optuna (Trial #20)*

7.2 Preprocessing Pipeline

Before model training, the following preprocessing steps were applied in sequence:

1. **Duplicate removal:** No duplicate rows were present in the dataset.
2. **Temporal feature extraction:** Transaction year, month, and quarter were extracted from the raw `transactiondate` column.
3. **High-missingness column removal:** Thirteen columns with more than 98% missing values were dropped.
4. **Target and identifier separation:** The `logerror` target and identifier column `parcelid` were isolated prior to feature engineering.

5. **Train/test split:** An 80/20 stratified split was applied with `random_state=42`, yielding 134,310 training and 33,578 test samples.
6. **Frequency encoding:** High-cardinality categorical columns (`propertycountylandusecode`, `propertyzoningdesc`) were replaced with their relative frequency in the training set.
7. **Numeric pipeline:** Median imputation followed by standard scaling (`StandardScaler`).
8. **Categorical pipeline:** Mode imputation followed by one-hot encoding (`OneHotEncoder` with `handle_unknown="ignore"`) for low-cardinality features (`hashottuborspa`, `taxdelinquencyflag`).

The final preprocessed feature matrix contains **49 features** (47 numeric and 2 binary one-hot encoded).

7.3 Final Model Performance

The final Random Forest was retrained on the full training set using the optimal hyperparameters from Optuna and evaluated on the held-out test set.

Model	**MAE**	**RMSE**	R^2
Baseline RF (default, $T = 100$)	0.075422	0.170595	-0.014465
Optuna-tuned RF ($T = 300$, tuned)	**0.069749**	**0.168521**	**0.010057**
Improvement	$+7.52\%$	$+1.22\%$	—

Table 4: *Comparison of Baseline and Optuna-Tuned Random Forest*

The Optuna-tuned model achieves a **7.52% reduction in MAE** and a **1.22% reduction in RMSE** over the baseline, while simultaneously flipping the R^2 coefficient from negative to positive (from -0.014 to $+0.010$), indicating that the tuned model is genuinely predictive relative to the trivial mean predictor.

8 Conclusion

8.1 Summary of Results

This report presented a complete machine learning pipeline for predicting the Zillow Zestimate log-error (`logerror`) using a Random Forest Regressor. The dataset comprised 167,888 merged property transaction records from 2016 and 2017 across Los Angeles County.

After preprocessing, a Random Forest baseline was established with default parameters, resulting with a test MAE of 0.075422. Hyperparameter optimisation via Optuna over 30 trials identified an improved configuration that reduced the test MAE to **0.069749**, representing a 7.52% improvement.

8.2 Future Work

After establishing the Random Forest as a baseline, our group's next phase of work will focus on broadening the model comparison to include additional machine learning models. The goal is to conduct benchmark study on the same preprocessing pipeline and evaluation protocol. The following models are planned for implementation in upcoming work:

1. **Logistic Regression**
2. **Support Vector Machine (SVM) Regressor**
3. **Deep Learning**

All three models will be evaluated under the same train/test splits and preprocessing pipelines, with performance reported using MAE, RMSE, and R^2 to compare against the Optuna-tuned Random Forest established in this report. This benchmark study will form the basis of the group's final project submission.